1-Introduction à la POO en PHP

La **Programmation Orientée Objet (POO)** consiste à organiser le code en **objets**.

Un objet = une instance d'une classe
Une classe = un modèle (comme un plan)

---

2. Structure d'une classe

```
class NomClasse {

    // 1. Attributs (Encapsulation)
    private $attribut1;
    private $attribut2;

    // 2. Constructeur
    public function __construct($a1, $a2) {
        $this->attribut1 = $a1;
        $this->attribut2 = $a2;
    }

    // 3. Méthodes NON statiques
    public function afficher() {
        echo $this->attribut1 . " " . $this->attribut2;
    }

    // 4. Méthode statique
    public static function info() {
        echo "Je suis une méthode statique";
    }
}
```

---

3. Encapsulation (private)

- `private` = accès uniquement dans la classe
- On utilise des getters/setters pour accéder

```
public function getAttribut1() {
    return $this->attribut1;
}

public function setAttribut1($val) {
    $this->attribut1 = $val;
}
```

---

4. Constructeur

Méthode appelée automatiquement lors de la création

```
public function __construct($nom) {
    $this->nom = $nom;
}
```

---

5. Instanciation (création d'objet)

```
$obj = new NomClasse("val1", "val2");
```

---

6. Appel des méthodes

```
$obj->afficher();        // méthode normale
NomClasse::info();       // méthode statique
```

---

7. Héritage

Une classe peut hériter d'une autre

```
class NCM {
    protected $nom;
}

class NCF extends NCM {
    public function afficherNom() {
        echo $this->nom;
    }
}
```

---

8. Polymorphisme

Une méthode peut être redéfinie

```
class Animal {
    public function parler() {
        echo "Je fais un bruit";
    }
}

class Chien extends Animal {
    public function parler() {
        echo "Wouf Wouf";
    }
}
```

EXEMPLE

Classe Stagiaire

```
class Stagiaire {
    private $cin;
    private $nom;
    private $ville;

    public function __construct($cin, $nom, $ville) {
        $this->cin = $cin;
        $this->nom = $nom;
        $this->ville = $ville;
    }

    public function afficher() {
        echo "CIN: $this->cin - Nom: $this->nom - Ville: $this->ville
<br>";
    }
}
```

Classe StagiairePro (héritage)

```
class StagiairePro extends Stagiaire {
    private $salaire;

    public function __construct($cin, $nom, $ville, $salaire) {
        parent::__construct($cin, $nom, $ville);
        $this->salaire = $salaire;
    }

    public function afficher() {
        parent::afficher();
        echo "Salaire: $this->salaire <br>";
    }
}
```

UTILISATION

```
$s1 = new Stagiaire("AA123", "Ali", "Casablanca");
$s1->afficher();

$s2 = new StagiairePro("BB456", "Sara", "Rabat", 5000);
$s2->afficher();
```

ATELIER 6 POO

Exercice 1 : Classe Voiture

❓ Énoncé

Créer une classe `Voiture` avec :

- marque
- modèle
- méthode afficher()

✅Correction

```
class Voiture {
    private $marque;
    private $modele;

    public function __construct($marque, $modele) {
        $this->marque = $marque;
        $this->modele = $modele;
    }

    public function afficher() {
        echo "Voiture: $this->marque $this->modele <br>";
    }
}
```

Exercice 2 : Méthode statique

❓ Énoncé

Ajouter une méthode statique qui affiche :
"Bienvenue dans le garage"

✅Correction

```
public static function message() {
    echo "Bienvenue dans le garage <br>";
}
```

Appel :

```
Voiture::message();
```

Exercice 3 : Héritage

❓ Énoncé

Créer une classe `VoitureSport` qui hérite de `Voiture`
Ajouter :

- vitesseMax

✅Correction

```
class VoitureSport extends Voiture {
    private $vitesseMax;

    public function __construct($marque, $modele, $vitesseMax) {
        parent::__construct($marque, $modele);
        $this->vitesseMax = $vitesseMax;
    }

    public function afficher() {
        parent::afficher();
        echo "Vitesse Max: $this->vitesseMax km/h <br>";
    }
}
```

Exercice 4 : Polymorphisme

❓ Énoncé

Créer :

- classe Employe → salaire()
- classe Manager → salaire() (plus élevé)

✅Correction

```
class Employe {
    public function salaire() {
        echo "Salaire de base <br>";
    }
}

class Manager extends Employe {
    public function salaire() {
        echo "Salaire élevé <br>";
    }
}
```

Exercice 5

Créer :

- classe Personne (nom, age)
- classe Etudiant (hérite + note)

✅Correction

```
class Personne {
    protected $nom;
    protected $age;

    public function __construct($nom, $age) {
        $this->nom = $nom;
        $this->age = $age;
    }

    public function afficher() {
        echo "$this->nom - $this->age ans <br>";
    }
}

class Etudiant extends Personne {
    private $note;

    public function __construct($nom, $age, $note) {
        parent::__construct($nom, $age);
        $this->note = $note;
    }

    public function afficher() {
        parent::afficher();
        echo "Note: $this->note <br>";
    }
}
```

Résumé simple

Concept	Explication
Classe	Modèle
Objet	Instance
private	Protection
constructeur	Initialisation
static	Sans objet
héritage	extends
polymorphisme	redéfinir méthode

Exercice 6 : Tableau d'objets Étudiant

Objectif

Manipuler :

- des objets
- un tableau d'objets
- une boucle
- appel de méthodes

❓ Énoncé

1. Créer une classe `Etudiant` avec :
   - nom
   - note
2. Créer une méthode :
   - `afficher()`
3. Créer **plusieurs objets Etudiant**
4. Les stocker dans un **tableau**
5. Parcourir le tableau pour afficher tous les étudiants
6. (Bonus □) : afficher uniquement les étudiants ayant une note ≥ 10

Correction complète

---

1. Classe Etudiant

```
class Etudiant {
    private $nom;
    private $note;

    public function __construct($nom, $note) {
        $this->nom = $nom;
        $this->note = $note;
    }

    public function afficher() {
        echo "Nom: $this->nom - Note: $this->note <br>";
    }

    public function getNote() {
        return $this->note;
    }
}
```

2. Création des objets

```
$e1 = new Etudiant("Ali", 12);
$e2 = new Etudiant("Sara", 8);
$e3 = new Etudiant("Youssef", 15);
$e4 = new Etudiant("Lina", 9);
```

3. Tableau d'objets

```
$etudiants = [$e1, $e2, $e3, $e4];
```

□ 4. Parcours du tableau

```
foreach ($etudiants as $etudiant) {
    $etudiant->afficher();
}
```

5. Bonus : étudiants admis (note ≥ 10)

```
echo "<h3>Étudiants admis :</h3>";

foreach ($etudiants as $etudiant) {
    if ($etudiant->getNote() >= 10) {
        $etudiant->afficher();
    }
}
```

Variante (niveau +)

Ajouter une fonction pour calculer la moyenne

```
function moyenne($tab) {
    $somme = 0;

    foreach ($tab as $etudiant) {
        $somme += $etudiant->getNote();
    }

    return $somme / count($tab);
}

echo "Moyenne = " . moyenne($etudiants);
```